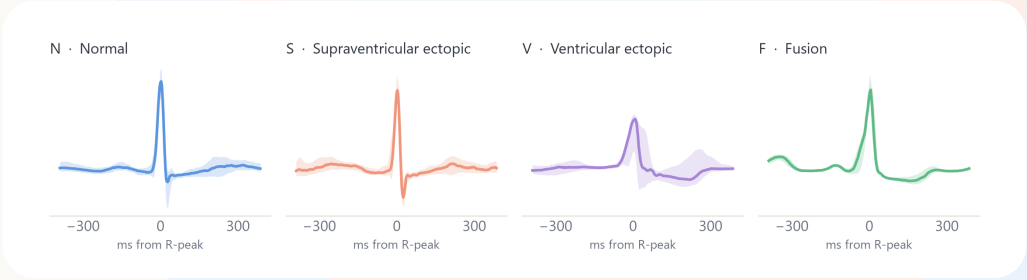
END-TO-END PROJECT WALKTHROUGH

PulseML

Teaching a computer to read heartbeats: data, code, models and results, explained step by step.



- Learn what the dataset is and why it is hard
- Read the real code, line by line
- Run every step yourself
- Understand why each model was chosen, and what the results mean

Research demonstration only. Not a medical device and not for clinical use.

ROADMAP

What's inside

1 The problem
What an ECG is, and the 4 kinds of heartbeat

2 The dataset
MIT-BIH: where the data comes from and its traps

3 Run it yourself
Setup and the 7 commands of the pipeline

4 Data pipeline code
Filtering, cutting beats, features, checks

5 The models
Why 4 models, and how each one works

6 Training
Class imbalance, augmentation, early stopping

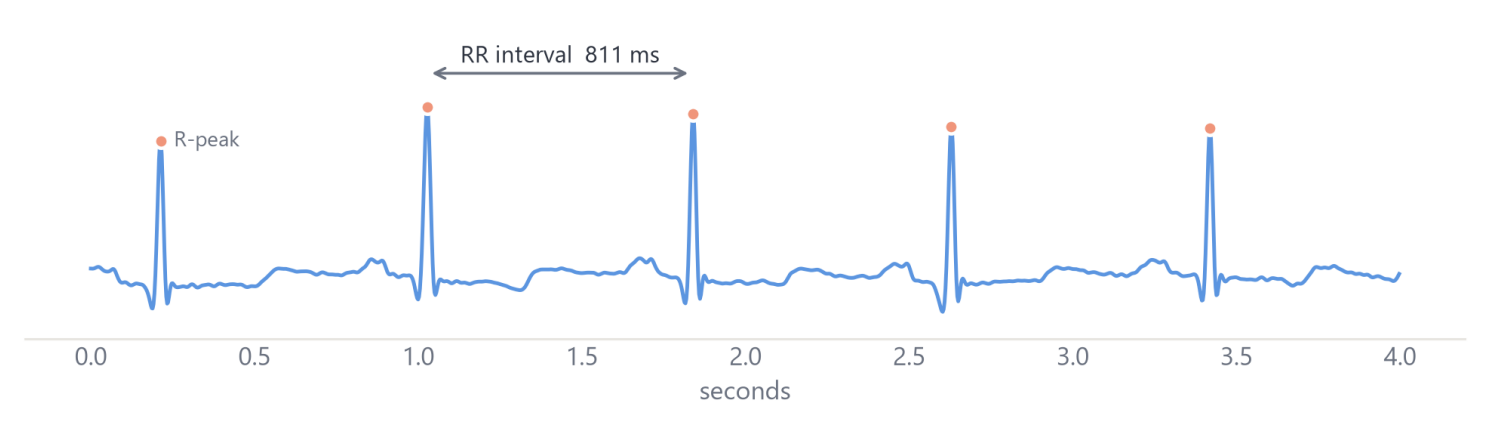
7 Results
What worked, what didn't, per class and per patient

8 Trust & serving
Calibration, prediction sets, API, dashboard

1 · THE PROBLEM

The problem: classify every heartbeat

An ECG records the heart's electricity. Each beat makes a spike (the R-peak). The gap between spikes is the RR interval.



Two kinds of clue tell beats apart:

- **Shape** (morphology): is the spike narrow and tall, or wide and odd?
- **Timing** (rhythm): did the beat arrive early compared with this person's usual rhythm?

The whole project is about combining these two clues well.

The task

Given one beat (0.78 s of signal) plus its timing, say which of 4 beat types it is. A doctor reviewing a 24-hour recording sees ~100,000 beats, so automatic sorting helps them find the rare abnormal ones.

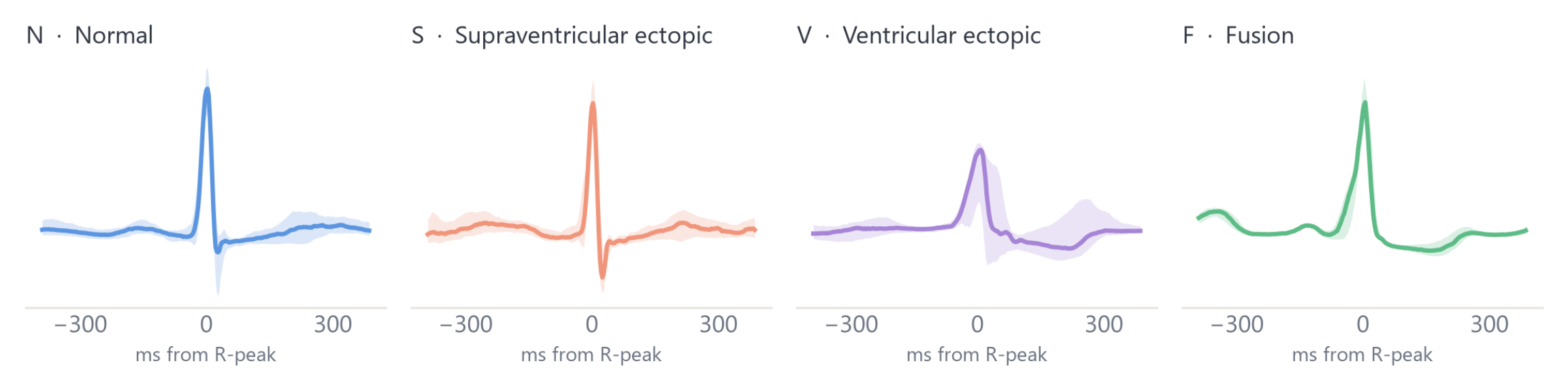
Why it is hard

~90% of beats are normal, abnormal beats look different in every person, and we must work on **people the model has never seen**.

1 · THE PROBLEM

The 4 beat classes (AAMI EC57 standard)

Typical shape of each class (median of real beats, shaded band = middle 50%). All beats are centred on the R-peak.



Class	Meaning	Where it starts	Main clue	Share of beats
N	Normal (incl. bundle branch block)	Normal pathway	Baseline	~89%
S	Supraventricular ectopic	Upper chambers, too early	Arrives early, normal shape	~3%
V	Ventricular ectopic	Lower chambers	Wide, odd shape, early	~7%
F	Fusion of V and normal	Both at once	Mix of N and V shapes	~0.8%

The dataset: MIT-BIH Arrhythmia Database

The standard public benchmark for heartbeat classification, freely available from PhysioNet.

48
half-hour recordings from 47 people (Boston, 1975–79)

360 Hz
samples per second, 2 leads (we use lead MLII)

100,630
labelled beats we extract and use

≥ 2
cardiologists annotated every single beat

What a record contains

- **.dat**: the raw voltage signal
- **.hea**: header (sampling rate, lead names, gain)
- **.atr**: the experts' annotations: the position of every R-peak and a symbol for its beat type (N, A, V, F …)

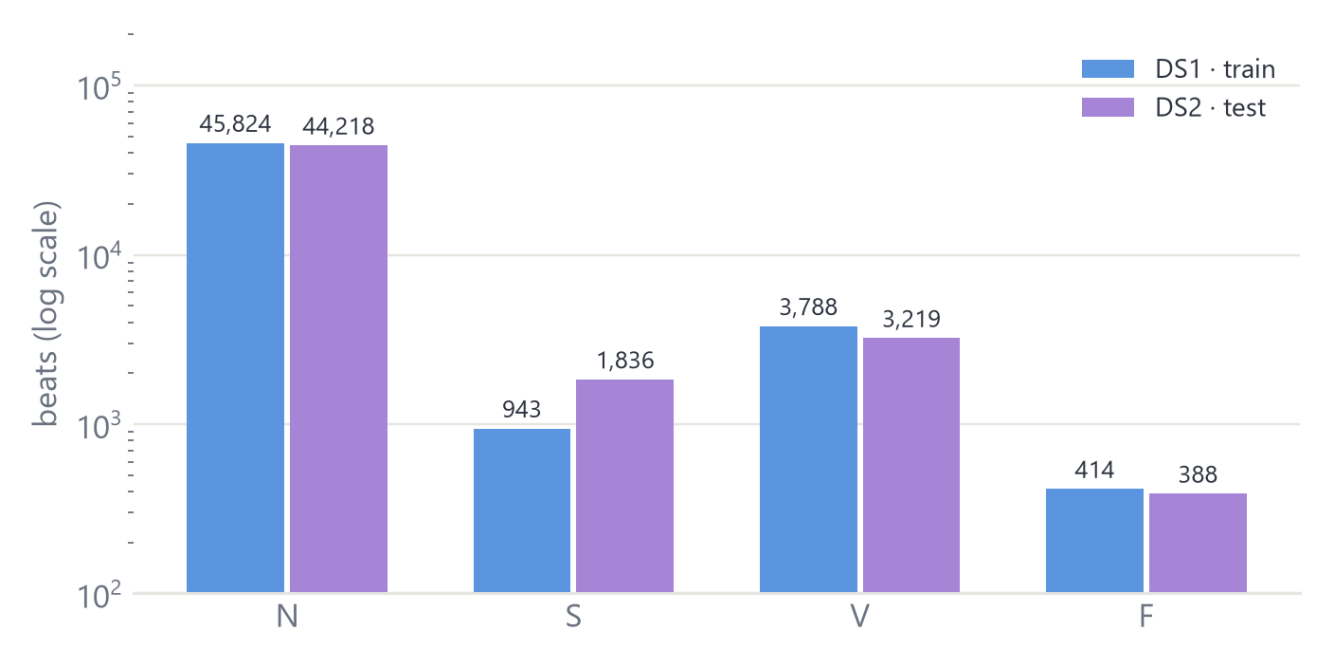
What we remove, and why

- **4 paced records** (102, 104, 107, 217): a pacemaker shapes those beats; the AAMI standard excludes them → 44 records
- **Unclassifiable beats** (Q) and non-beat notes (rhythm changes, noise marks)
- First/last beat of each record (no neighbour, so no RR interval)

Lineage: 103,724 annotations → 100,733 beat annotations → **100,630 beats** (15 excluded as paced/unclassifiable, 88 first/last beats). Every count is logged and checked in the data-quality report.

Trap #1: extreme class imbalance

Train set (DS1) and test set (DS2) beat counts. Note the log scale: F is ~100× rarer than N.



The accuracy trap

A lazy model that always answers "N" scores **89% accuracy** on DS2 while finding zero abnormal beats. So accuracy is never reported alone.

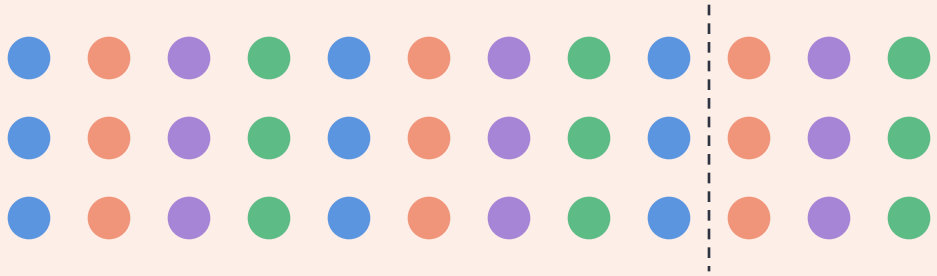
What we do about it

- Score models with **macro-F1** (every class counts equally)
- Weight the loss so mistakes on rare classes cost more
- Report recall and precision for each class separately

Trap #2: the same person must never be in train and test

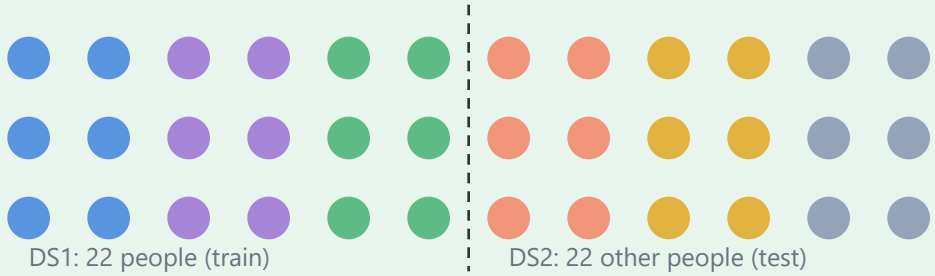
Beats from one person look alike. If they are shuffled into both sets, the model memorises people instead of learning beats.

Random beat split (the common mistake)



Colours = patients. Every patient appears on both sides of the line, so the test is easy. Papers that do this report **>99% accuracy** that collapses on new people.

Inter-patient split (what we do)



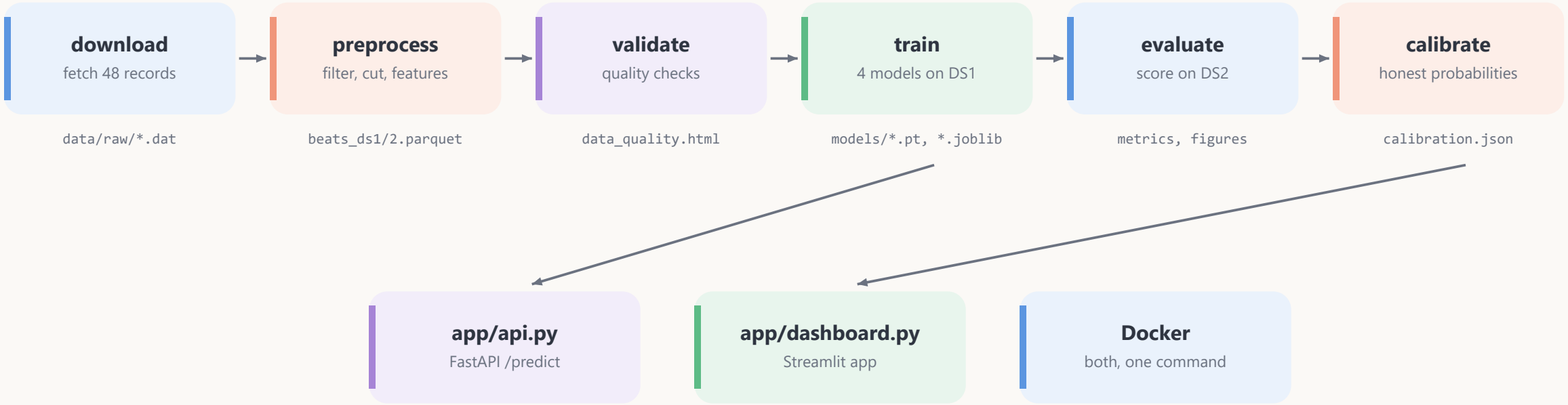
de Chazal et al. (2004) split: **no person is in both sets**. Scores are lower but honest. The code asserts this, and the validation step checks it again.

The same rule applies inside training: validation and cross-validation also hold out **whole patients**, never single beats.

3 · RUN IT YOURSELF

The pipeline at a glance

Each box is one Python module in src/ that you run with python -m src.<name>. Each step writes files the next step reads.



Optional research steps: **experiments** (ablation with 3 seeds) and **experiments --cv-select** (5-fold patient-grouped cross-validation that picked the final model).

Run it yourself: setup and commands

setup

```
# 1. get the code and a virtual environment
git clone <repo-url> pulseml && cd pulseml
python -m venv .venv
.venv\Scripts\activate        # Windows
pip install -r requirements.txt

# 2. (optional) GPU build of PyTorch
pip install torch --index-url \
  https://download.pytorch.org/whl/cu126
```

the pipeline, in order

```
python -m src.download      # ~100 MB, once
python -m src.preprocess    # ~1 min
python -m src.validate      # quality report
python -m src.train         # ~2 min on GPU
python -m src.evaluate      # DS2 scores + figures
python -m src.calibrate     # ~10 min on GPU
pytest tests/ -q            # 82 tests
```

run the apps

```
uvicorn app.api:app --reload    # localhost:8000/docs
streamlit run app/dashboard.py  # localhost:8501

docker compose up --build       # both, in containers
```

Good to know

Trained models and results are already in `outputs/`, so the API and dashboard work straight after cloning; you only need the pipeline to retrain. Training uses the GPU automatically if one is available.

Where everything lives

`src/config.py`	every constant: paths, record lists, filter settings, hyperparameters
`src/download.py`	fetches MIT-BIH from PhysioNet (skips files you already have)
`src/preprocess.py`	filter → find beats → label → cut windows → features → Parquet
`src/validate.py`	data-quality checks, DuckDB SQL summaries, HTML report
`src/dataset.py`	turns Parquet rows into PyTorch tensors; patient templates
`src/models.py`	LogReg, XGBoost, ECGCNN, HybridCNN, SeedEnsemble, calibration maps
`src/train.py`	patient-level splits, grid search, the training loop
`src/experiments.py`	ablation study and cross-validated model selection
`src/evaluate.py`	DS2 metrics, figures, model card
`src/calibrate.py`	calibrated probabilities + conformal prediction sets
`app/`	the web API (api.py) and the Streamlit dashboard (dashboard.py)
`scripts/`	builds this walkthrough PDF
`tests/`	82 automatic checks (run with pytest)

4 · DATA PIPELINE

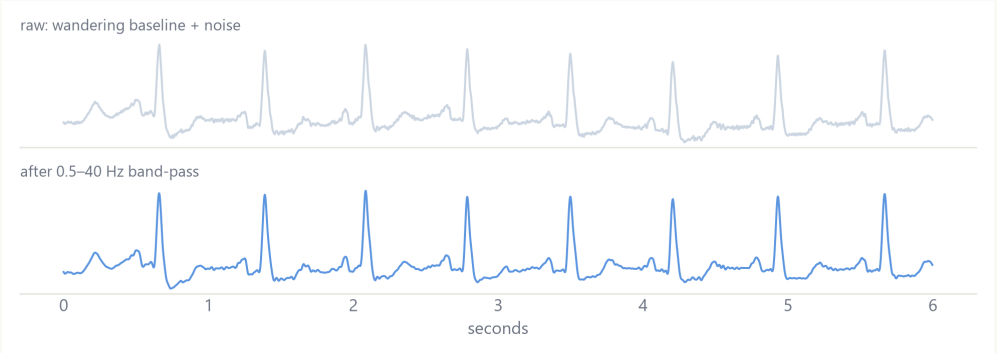
Step 1 · Clean the signal (band-pass filter)

Breathing makes the line drift slowly; muscles and mains power add fast fuzz. We keep only 0.5–40 Hz.

src/preprocess.py

```python
def bandpass_filter(x, fs=config.FS, low=config.BANDPASS_LOW, high=config.BANDPASS_HIGH,
        order=config.BANDPASS_ORDER):
    sos = sps.butter(order, [low, high], btype="bandpass", fs=fs, output="sos")
    return sps.sosfiltfilt(sos, np.asarray(x, dtype=np.float64))  # zero-phase, so R-pea
      ks don't move
```

1. **x** is the raw voltage. The defaults come from config: 360 Hz sampling, keep 0.5–40 Hz, filter order 4.
2. Design a **Butterworth** band-pass filter (smooth, no ripples). **sos** = second-order sections, the numerically stable form.
3. **sosfiltfilt** runs the filter forward and then backward, which cancels the time delay (zero phase). The R-peak stays on exactly the same sample.



Step 2 · Cut one window per beat and normalise it

Each beat becomes 280 numbers: 140 samples before the R-peak and 140 after (0.78 s), rescaled to mean 0 and spread 1.

src/preprocess.py

```
1  def extract_window(x, r_peak, before=config.WINDOW_BEFORE, after=config.WINDOW_AFTER
       ):
2      start, end = r_peak - before, r_peak + after
3      return x[start:end] if start >= 0 and end <= len(x) else None
4
5  def zscore(w, eps=1e-8):
6      w = np.asarray(w, dtype=np.float64)
7      return (w - w.mean()) / (w.std() + eps)
```

1–2 Window = from 140 samples before the R-peak to 140 after it.

3 If the window would run past the start or end of the recording, return **None** and skip the beat. We never pad with made-up signal.

5–6 **zscore**: convert to floating point.

7 Subtract the mean and divide by the standard deviation. Every beat then has mean 0 and spread 1, which removes gain differences between people and devices. The tiny **eps** stops a flat line from dividing by zero.

Why normalise? One patient's electrodes may give 2 mV spikes, another's 0.8 mV. Without z-scoring the model would learn "big = person A" instead of learning beat shapes.

Step 3 · Turn one recording into labelled beats

src/preprocess.py · process_signal (first part)

```python
def process_signal(sig_mv, samples, symbols, record_id, fs=config.FS):
    """Cut one filtered record into labelled beats. Returns (rows, lineage counts)."""
    symbols = list(symbols)
    is_beat = np.array([s in BEAT_SYMBOLS for s in symbols], dtype=bool)
    peaks = np.asarray(samples)[is_beat]
    beat_syms = [s for s, b in zip(symbols, is_beat) if b]
    ann_idx = np.flatnonzero(is_beat)
    rr = np.diff(peaks).astype(np.float64)  # rr[k-1] is the interval ending at beat k

    rows, counts = [], Counter()
    for k, (r, sym) in enumerate(zip(peaks, beat_syms)):
        label = map_aami(sym)
        if label is None:
            counts["excluded_class"] += 1
            continue
        if k in (0, len(peaks) - 1):
            counts["excluded_no_rr"] += 1
            continue
        win = extract_window(sig_mv, int(r))
        if win is None:
            counts["excluded_boundary"] += 1
            continue
        local_avg = rr[max(0, k - config.RR_LOCAL_WINDOW) : k].mean()  # causal: only past intervals
        rows.append(
            {
                "record_id": record_id,
                "beat_idx": int(ann_idx[k]),
                "r_peak_sample": int(r),
                "symbol": sym,
                "label": label,
                "waveform": zscore(win).astype(np.float32),
                **beat_features(win, float(r - peaks[k - 1]), float(peaks[k + 1] - r), fs, float(local_avg)),
            }
        )
        counts[config.CLASS_NAMES[label]] += 1
```

4 Mark which annotations are beats (others are notes such as rhythm changes).

5–7 Keep the R-peak positions and symbols of the beats.

8 RR intervals = gaps between neighbouring R-peaks.

12–15 Map the symbol to N/S/V/F. Paced or unknown beats are dropped, but still count for timing.

16–18 First and last beat have no neighbour, so no RR: drop.

19–22 Cut the 280-sample window; skip it if it runs off the edge.

23 This patient's usual RR = mean of the **previous** 32 intervals only (no peeking ahead).

24–34 One row per beat: ids, label, z-scored waveform and 16 features.

35 Count beats per class for the lineage report.

Step 4 · Timing features, relative to each patient

A heart rate of 90 bpm is normal for one person and fast for another, so timing is compared with the patient's own rhythm.

src/preprocess.py

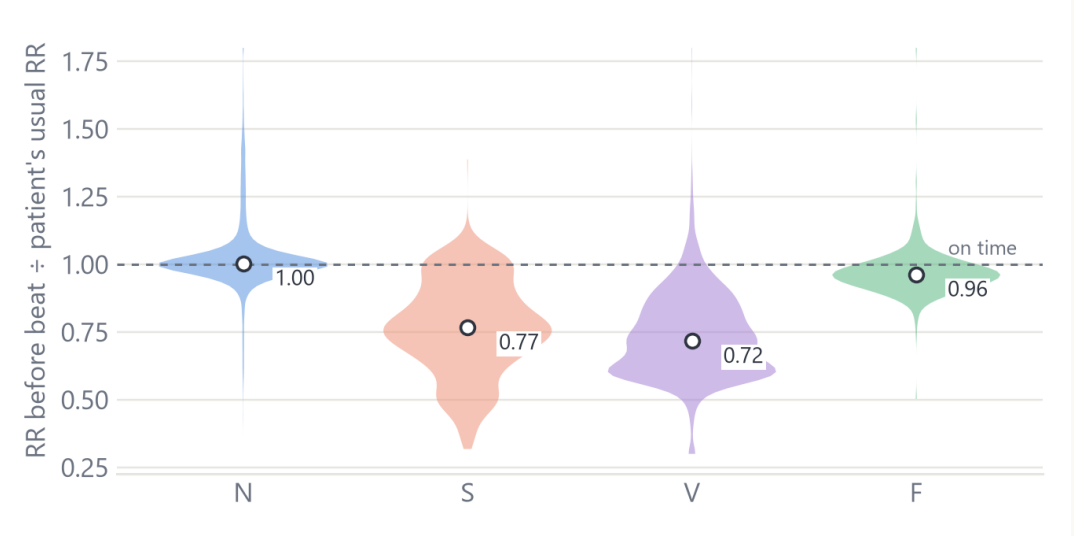
```python
def rr_features(rr_prev, rr_next, rr_local_avg=None, fs=config.FS):
    """RR features (in samples). Also used by the API so serving matches training.
      """
    avg = float(rr_prev if rr_local_avg is None else rr_local_avg)
    return {
        "rr_prev": float(rr_prev),
        "rr_next": float(rr_next),
        "rr_ratio": _div(rr_prev, rr_next),
        "local_hr": _div(fs * 60.0, rr_prev),
        "rr_local_avg": avg,
        "rr_prev_norm": _div(rr_prev, avg),
        "rr_next_norm": _div(rr_next, avg),
    }
```

3 No history yet? Use rr_prev as the patient's usual RR.

7–8 Ratio of the two gaps, and heart rate in beats per minute (360 × 60 ÷ RR).

10 **rr_prev_norm** < 1 means the beat came early for this person: the key clue for S beats.

11 **rr_next_norm** > 1 means a pause after the beat (typical after V).



Normal beats arrive on time (1.00). S and V beats arrive ~25% early. Adding these 3 patient-relative features raised logistic regression from 0.51 to **0.59** macro-F1 on unseen patients.

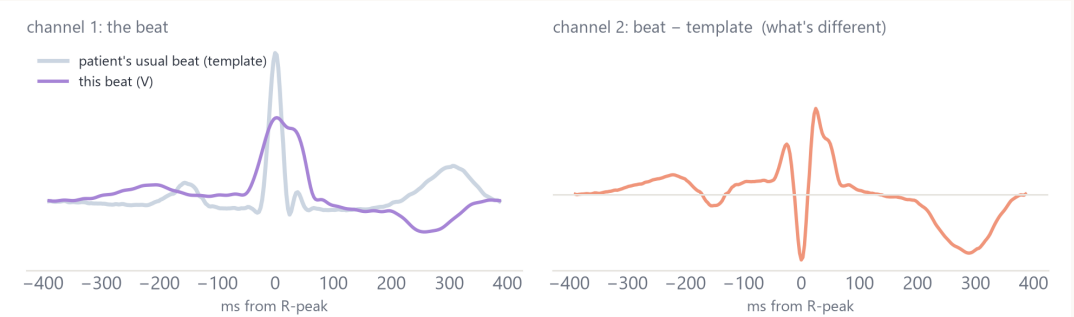
Step 5 · Shape features and the patient template

Besides timing, 9 numbers describe the shape. The deep model also gets the patient's own 'usual beat' to compare against.

Feature	What it measures
wf_mean, wf_std, wf_min, wf_max	Level, spread and extremes of the (un-normalised) beat in mV
wf_skew, wf_kurtosis	Lopsidedness and peakiness of the shape
qrs_proxy	Tallest point in the middle 40 samples (the QRS spike)
pre_energy, post_energy	Signal energy before the spike (P-wave) and after it (T-wave)

src/dataset.py

```
1  def template_matrix(df, k=config.TEMPLATE_BEATS):
2      """Median of the previous k beats of the same record (causal, label-free).
3      The first beat of a record is its own template."""
4      wf = waveform_matrix(df)
5      out = np.empty_like(wf)
6      rec, beat = df["record_id"].to_numpy(), df["beat_idx"].to_numpy()
7      for r in pd.unique(rec):
8          idx = np.flatnonzero(rec == r)
9          idx = idx[np.argsort(beat[idx], kind="stable")]
10         w = wf[idx]
11         out[idx[0]] = w[0]
12         for i in range(1, len(w)):
13             out[idx[i]] = np.median(w[max(0, i - k) : i], axis=0)
14     return out
```



7–9 For each patient, take their beats in time order.

11 The first beat has no history, so it is its own template.

12–13 Template = sample-by-sample **median of the previous 16 beats**. Only the past is used, so this works live.

The median ignores the odd abnormal beat, so the template is the person's typical normal beat. Beat − template highlights exactly what is unusual.

4 · DATA PIPELINE

Step 6 · Check the data before training

python -m src.validate runs automatic checks and writes outputs/reports/data_quality.html. All checks pass.

Missing values no NaN in any column or waveform	0	**Amplitude** beats outside ±5 mV (warning only)	0
Flat signal lead-off / all-zero beats	0	**Window length** every waveform exactly 280 samples	0
Duplicates same (record, beat) key twice	0	**Label validity** labels outside 0–3	0
RR > 3 s long gaps: real pauses (record 232) or unreadable stretches	80 flagged	**Patient leakage** a record in both DS1 and DS2	none

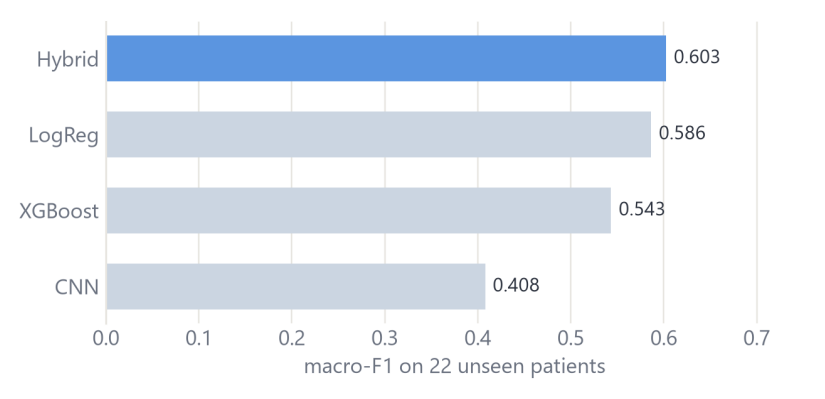
The checks are plain functions on a DataFrame (check_missing, check_flat, …), so the tests feed them deliberately broken data (all-zero beats, 12 mV spikes, duplicates) and confirm each one is caught. DuckDB SQL produces the per-record summary tables in the report.

5 · THE MODELS

Why these 4 models?

Each model answers a question. Together they show which ingredients actually matter on new patients.

Model	What it looks at	Why it is here	Macro-F1 (DS2)
Logistic regression	16 hand-made features	Simplest sensible baseline: can good features alone do the job? Fast and easy to explain.	0.586
XGBoost	Same 16 features	Decision trees catch non-linear combinations ("early AND wide"). Standard strong choice for tables.	0.543
1D CNN (specified)	Raw beat shape only	Learns shape patterns by itself, with no hand-made features. But it cannot see timing.	0.408
Hybrid-rel ensemble	Shape + difference from the patient's usual beat + patient-relative timing	Combines both clues, each compared with the patient's own baseline. 3 copies vote.	**0.603**



Lesson: the gain came from giving the model the right context (timing and the patient's own normal beat), not from a bigger model. XGBoost even kept choosing its smallest trees: with only 22 training people, simpler generalises better.

Baselines: logistic regression and XGBoost

src/models.py

```python
def make_logreg(C=1.0, seed=config.SEED):
    clf = LogisticRegression(C=C, class_weight="balanced", max_iter=2000, random_state=seed)
    return Pipeline([("scaler", StandardScaler()), ("clf", clf)])

def make_xgb(seed=config.SEED, **params):
    defaults = dict(
        objective="multi:softprob", num_class=config.NUM_CLASSES, tree_method="hist", subsample=0.8,
        colsample_bytree=0.8, eval_metric="mlogloss", random_state=seed, n_jobs=-1,
    )  # fmt: skip
    return XGBClassifier(**(defaults | params))
```

2 **LogisticRegression** learns one weight per feature per class. **class_weight="balanced"** makes rare classes count more. **C** controls how simple the model stays.

3 **StandardScaler** first puts every feature on the same scale.

6–10 **XGBoost**: hundreds of small decision trees, each fixing the previous ones' mistakes. Rows and columns are subsampled (0.8) to reduce overfitting.

How the settings were chosen

GridSearchCV tries every combination (LogReg C in {0.01, 0.1, 1, 10}; XGBoost depth, trees, learning rate) with 3-fold cross-validation **grouped by patient**, scored by macro-F1, on DS1 only.

Winners: LogReg C = 0.01 (strongest simplicity), XGBoost depth 2, 100 trees, rate 0.05.

Strengths and weaknesses

LogReg: best N/V balance among baselines (macro-F1 0.586).
XGBoost: best macro-AUC of the baselines (0.941) and catches 93% of S beats, but raises many false alarms on normal beats (N recall 0.76).

The 1D CNN: learning shapes directly

A convolution slides small learned pattern-detectors along the beat; deeper layers combine them into bigger patterns.

src/models.py

```python
def conv_block(c_in, c_out, k, pool=True):
    layers = [nn.Conv1d(c_in, c_out, k, padding=k // 2), nn.BatchNorm1d(c_out), nn.ReLU(inplace=True)]
    return layers + [nn.MaxPool1d(2) if pool else nn.AdaptiveAvgPool1d(1)]

def conv_trunk(in_channels=1):
    # (batch, C, 280) -> (batch, 128, 1)
    return nn.Sequential(*conv_block(in_channels, 32, 7), *conv_block(32, 64, 5), *conv_block(64, 128,
      3, pool=False))
```

2 One block = **Conv1d** (k-sample pattern detectors) → **BatchNorm** (keeps numbers in a stable range) → **ReLU** (keeps only positive evidence). padding = k // 2 keeps the length.

3 **MaxPool** halves the length (keeps the strongest response). The last block instead averages over the whole beat.

5–7 Three blocks: 32 detectors of width 7 → 64 of width 5 → 128 of width 3. Output: 128 numbers describing the beat.



44,228 learned numbers (parameters).

Dropout (50% then 30%) randomly switches off parts while training so the network can't memorise.

Result: macro-F1 0.408. Shape alone is not enough: the CNN cannot tell an early-but-normal-looking S beat from a normal one, and it found almost no F beats.

The recommended model: HybridCNN-rel

Two inputs, two branches, one decision. The beat goes in as 2 channels: the beat itself, and beat − patient template.

src/models.py · HybridCNN

```python
class HybridCNN(nn.Module):
    """Conv trunk on the beat + small MLP on the RR features.

    A single beat can't tell S from N; timing relative to the patient's rhythm can. The RR
    normalisation stats live in buffers so the checkpoint takes raw feature values.
    """

    def __init__(self, n_features=len(config.RR_FEATURES), n_classes=config.NUM_CLASSES, in_channels=1):
        super().__init__()
        self.in_channels = in_channels
        self.features = conv_trunk(in_channels)
        self.rr_branch = nn.Sequential(
            nn.Linear(n_features, 32),
            nn.BatchNorm1d(32),
            nn.ReLU(inplace=True),
            nn.Linear(32, 32),
            nn.ReLU(inplace=True),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5), nn.Linear(128 + 32, 64), nn.ReLU(inplace=True), nn.Dropout(0.3), nn.Linear(64, n_cla
  sses)
        )
        self.register_buffer("feat_mean", torch.zeros(n_features))
        self.register_buffer("feat_std", torch.ones(n_features))

    def set_feature_stats(self, mean, std):
        self.feat_mean.copy_(torch.as_tensor(mean, dtype=torch.float32))
        self.feat_std.copy_(torch.as_tensor(std, dtype=torch.float32).clamp(min=1e-6))

    def forward(self, x, feats=None):
        if feats is None:
            raise ValueError("HybridCNN needs RR features")
        z = (feats - self.feat_mean) / self.feat_std
        return self.classifier(torch.cat([self.features(x).flatten(1), self.rr_branch(z)], dim=1))
```

11 The same conv trunk as the CNN; with in_channels = 2 it reads beat and beat − template.

12–18 A small network for the 7 timing features → 32 numbers.

19–21 Join 128 shape + 32 timing numbers → 64 → 4 class scores.

22–23 The timing features' mean and spread (from training data) are stored **inside** the model file, so the API can't scale them differently.

29–33 Forward pass: standardise timing, run both branches, concatenate, classify.

143,628 parameters in total: 3 copies (seeds 42, 7, 2024) averaged by **SeedEnsemble**, which reduces luck from random initialisation.

Training (1/3): making rare classes count

src/train.py

```
1 def class_weights(counts):
2     # inverse frequency, scaled so a balanced dataset gets all ones
3     counts = np.maximum(counts, 1).astype(np.float64)
4     return torch.tensor(counts.sum() / (len(counts) * counts), dtype=torch.f
        loat32)
```

3 Avoid dividing by zero if a class is missing.

4 weight = total ÷ (4 × count of that class). Rare class → big weight. A perfectly balanced dataset would give weight 1 for every class.

0.28
weight of N

13.51
weight of S

3.36
weight of V

30.78
weight of F

A missed F beat costs ~110× more than a missed N beat. Without this, the network learns to say "N" all the time.

Data augmentation (training only)

Each batch gets small random changes that keep the label true:
• shift the beat by up to ±5 samples (R-peak not perfectly centred)
• gain ×0.85–1.15 (stronger/weaker electrodes)
• a little noise and slow baseline drift
This imitates differences between people and devices.

Optimiser

Adam, learning rate 0.001, weight decay 0.0001 (gently pulls weights toward zero), batches of 256 beats.

Training (2/3): the training loop

src/train.py · one pass over the data (an epoch)

```python
def run_epoch(model, loader, criterion, device, optimizer=None, augment=False, gen=None):
    training = optimizer is not None
    model.train(training)
    total_loss, preds, labels = 0.0, [], []
    with torch.set_grad_enabled(training):
        for wf, feat, y in loader:
            if training and augment:
                wf = augment_batch(wf, gen)
            wf, feat, y = wf.to(device), feat.to(device), y.to(device)
            logits = model(wf, feat)
            loss = criterion(logits, y)
            if training:
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * len(y)
            preds.append(logits.argmax(1).cpu().numpy())
            labels.append(y.cpu().numpy())
    p, t = np.concatenate(preds), np.concatenate(labels)
    f1 = f1_score(t, p, average="macro", labels=list(range(config.NUM_CLASSES)), zero_division=0)
    return {"loss": total_loss / len(t), "acc": accuracy_score(t, p), "macro_f1": float(f1)}
```

- **2–3** Training mode if an optimiser is given (dropout on), otherwise evaluation mode.
- **5** Only track gradients while training (saves memory when evaluating).
- **6** Loop over batches of 256 beats: waveform, timing features, label.
- **7–8** Augment the batch (training only).
- **10–11** Predict class scores (logits) and measure how wrong they are (weighted cross-entropy).
- **13–15** Back-propagation: work out how each weight should change, then nudge it.
- **19–21** Summarise the epoch: loss, accuracy and macro-F1.

Training (3/3): when to stop

Train too little and the model is vague; too long and it memorises the training patients. Held-out patients decide.



Notice the gap: training F1 keeps rising while validation F1 stalls. That gap is the model starting to memorise the 18 training patients instead of learning beats that generalise.

Early stopping (CNN and ablations)

After every epoch, measure macro-F1 on held-out patients. Keep the best weights; stop after 7 epochs with no improvement.

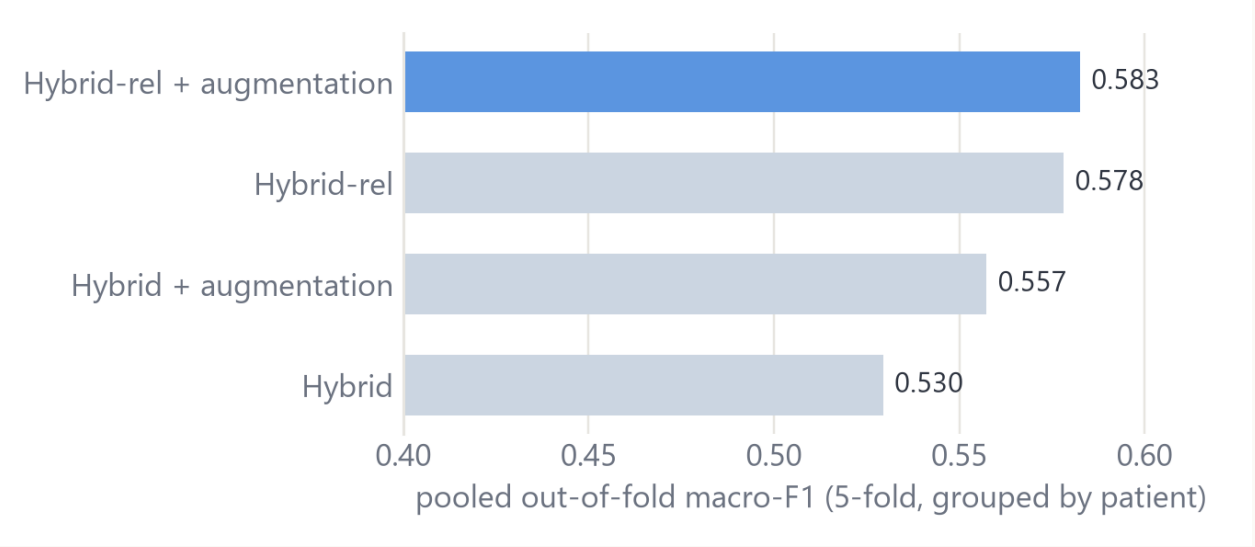
Final hybrid model

Cross-validation showed ~7 epochs is best, so it is retrained on **all 22** training patients for exactly 7 epochs, 3 times with different seeds, and the 3 copies are averaged.

6 · TRAINING

Choosing the final model fairly

Validation on only 4 patients was too noisy to separate the best variants, so the final choice used 5-fold cross-validation.



- Split the 22 training patients into 5 groups. Train on 4 groups, predict the 5th, rotate.
- Pool all out-of-fold predictions and compute one macro-F1 per variant.
- **Winner: Hybrid-rel + augmentation (0.583)**, the variant now in the API.
- The test patients (DS2) were never used to choose anything.

Honesty note: the switch from the 4-patient validation split to cross-validation was made after seeing that validation and test disagreed. It is documented in docs/EXPERIMENTS.md. Seed-to-seed differences (±0.02–0.04 macro-F1) are as big as the gaps between top variants, which is why every comparison uses 3 seeds or 5 folds.

How to read the scores

Recall (sensitivity)

Of all real S beats, what share did we find? Missing abnormal beats is the dangerous error.

Precision

Of all beats we called S, what share really were S? Low precision = many false alarms.

F1

One number balancing precision and recall (their harmonic mean). 1 is perfect.

Macro-F1

Average F1 over N, S, V and F with equal weight. Our main score, because it can't be gamed by predicting N.

ROC-AUC

Chance that a random abnormal beat gets a higher score than a random normal one. 0.5 = guessing, 1 = perfect ranking.

Specificity

Of all beats that are NOT S, what share did we correctly leave alone?

7 · RESULTS

Results on 22 people the model never saw

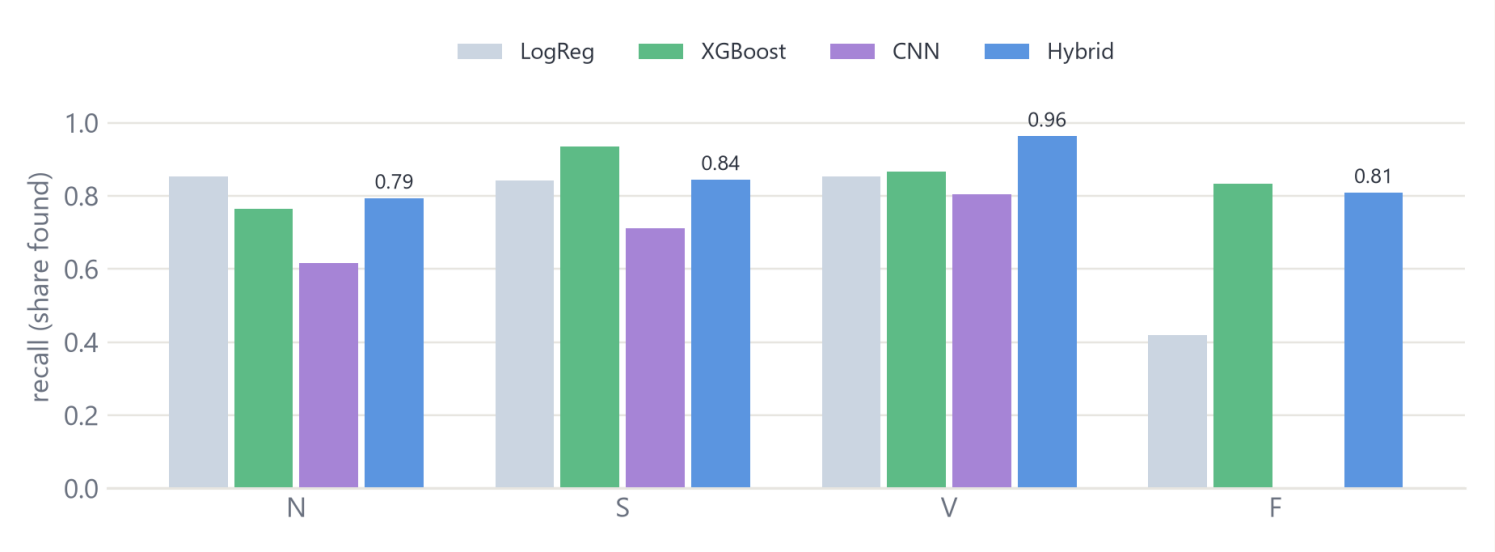
DS2: 49,661 beats. Hybrid-rel ensemble compared with the three other models.

0.603
macro-F1 (best of all models)

0.963
macro ROC-AUC (ranking quality)

96%
of V beats found

84%
of S beats found

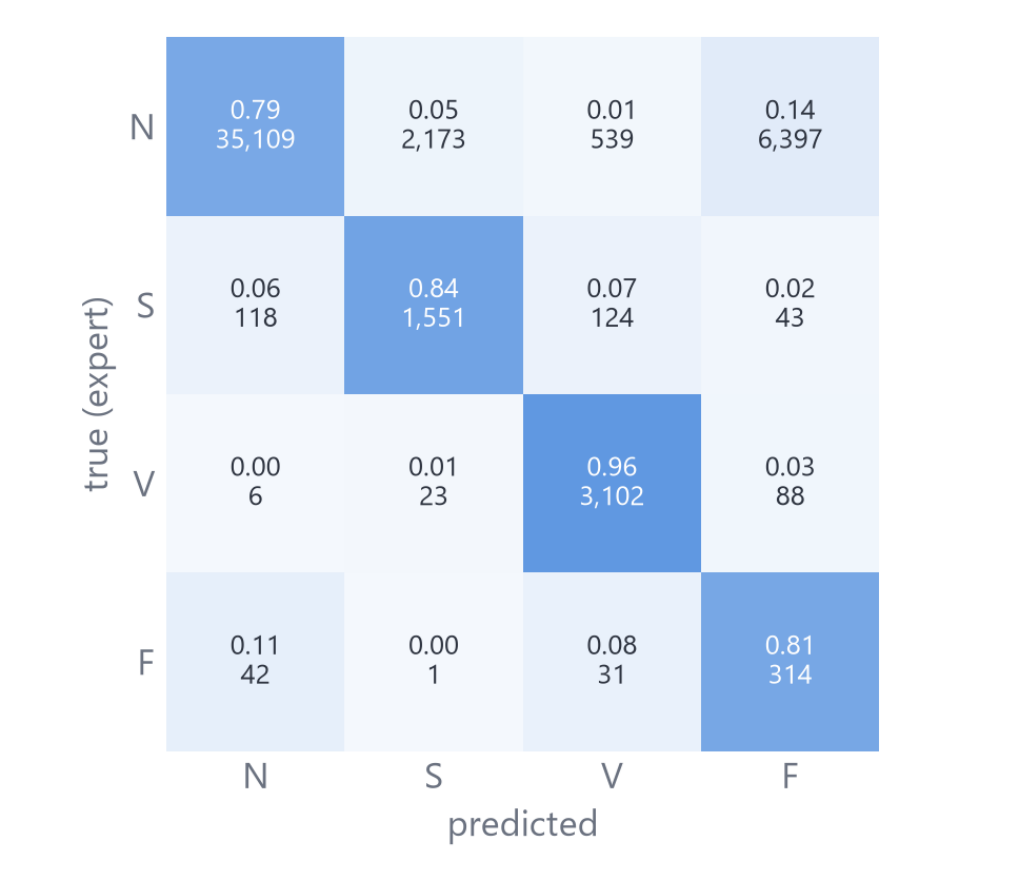


Read the chart: each group is a beat class, each bar a model. The blue Hybrid is the most balanced: best on V, near the top on S and F, without XGBoost's many false alarms on N.

The CNN finds no F beats at all (bar at zero).

Where the Hybrid model goes wrong

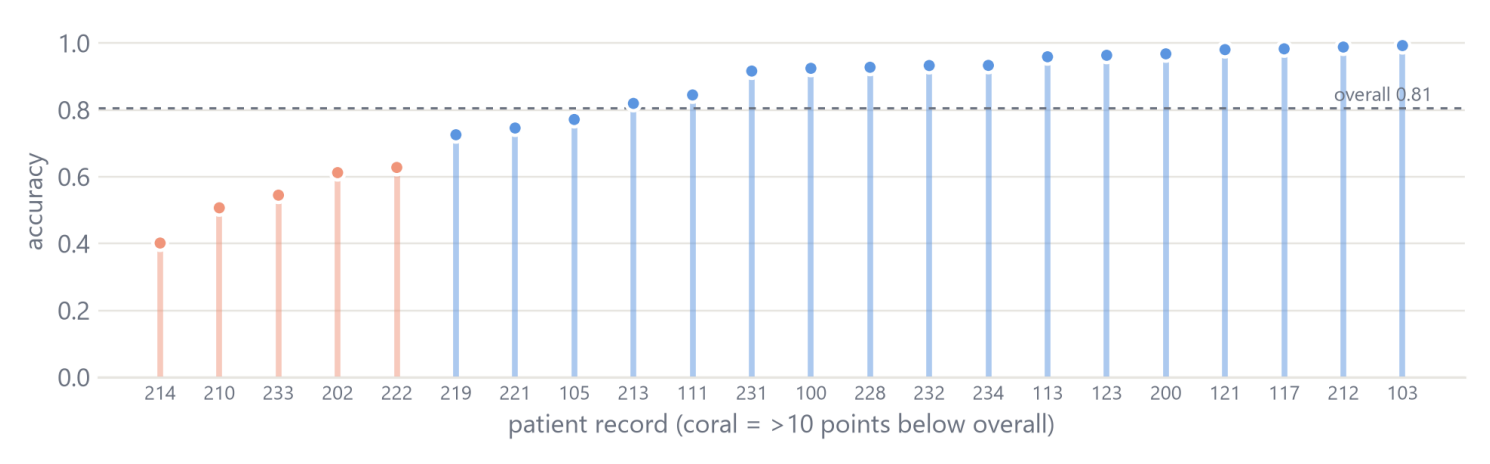
Confusion matrix: rows = expert label, columns = model's answer. Each row adds up to 1. Diagonal = correct.



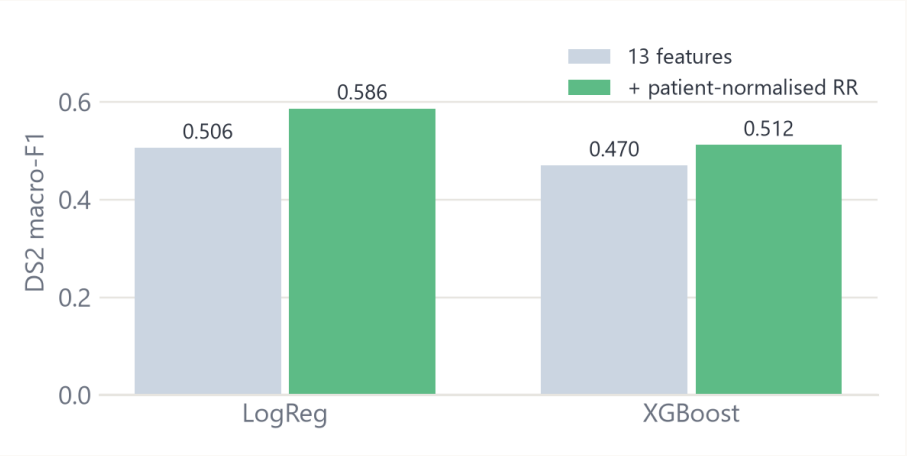
- **V is excellent**: 96% found, and few other beats are mistaken for V.
- **S is found (84%) but over-called**: 5% of normal beats (2,173) are flagged S. Because normal beats are so common, more S alarms are false than true (precision 0.41).
- **F is the weakest class**: 81% of F beats are found, but 14% of normal beats (6,397) are also called F, so almost every F alarm is false (precision 0.05). Nearly all training F beats come from one patient (record 208).
- In a real workflow these false alarms are reviewed by a human; missing a dangerous beat is worse than a false alarm.

Results differ from person to person

Per-patient accuracy of the Hybrid model. Averages hide this, so every evaluation also reports each record.



5 of 22 patients are more than 10 points below the overall accuracy (coral). They have unusual baselines or many borderline beats. This is the main remaining weakness, and the motivation for patient-specific adaptation in future work.



Ablation = remove one ingredient and measure the drop. Adding the 3 patient-normalised timing features was the single biggest win for logistic regression (0.506 → 0.586).

Can we trust the percentages? Calibration

Rare-class weighting inflates the raw S and F percentages. Calibration rescales them so that a stated 80% is right about 80% of the time.

src/models.py

```python
def apply_calibration(logp, temperature=1.0, bias=None):
    """softmax(logp / T + b)"""
    z = np.asarray(logp, dtype=np.float64) / temperature
    if bias is not None:
        z = z + np.asarray(bias, dtype=np.float64)
    e = np.exp(z - z.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)
```



3 Divide the log-probabilities by a **temperature** T (T > 1 softens over-confident answers).

4–5 Add a small **bias per class** (BCTS): undoes the push toward rare classes from class weighting.

6–7 Softmax turns the scores back into probabilities that add up to 1.

T and the biases are fitted on **training-set out-of-fold predictions only**. On unseen patients the log-loss drops from 0.477 to 0.158. Labels still come from the raw model, because the calibrated argmax would almost never say S.

8 · TRUST & SERVING

Prediction sets: "it's N or S"

Instead of one answer, return every class that is still plausible, with a target of containing the truth 90% of the time.

src/calibrate.py

```python
def conformal_qhat(probs, y, alpha=ALPHA):
    """Per-class conformal threshold on the score 1 - p_true."""
    qhat = []
    for k in LABELS:
        s = np.sort(1.0 - probs[y == k, k])
        n = len(s)
        qhat.append(float(s[min(n, math.ceil((n + 1) * (1 - alpha))) - 1]) if n el
      se 1.0)
    return qhat
```

4 One threshold per class ("Mondrian" conformal), so rare classes get their own guarantee.

5 Score = 1 − probability given to the true class: how surprised the model was.

7 Take the score below which 90% of that class's examples fall.

Rule at prediction time

Class k goes in the set if $1 - p_k \le \text{threshold}_k$.
One class in the set = confident. Two or three = uncertain, so flag the beat for a human.

How well it worked

On training patients (out-of-fold): exactly 90% for every class.
On new patients: 86% overall (S/V/F ≈ 98%, N 85%), average set size 1.61.
The guarantee weakens because new people are not exactly like the training people. This is reported, not hidden.

Serving: the prediction API

app/api.py · predict (first half; then calibration, logging, response)

```python
@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    require_model()
    x = np.asarray(req.waveform, dtype=np.float32)
    x = (x - x.mean()) / (x.std() + 1e-8)
    wf, feats, name, template_used = x[None, None], None, "cnn", False

    if req.rr_context and "hybrid" in models:
        name = "hybrid"
        rc = req.rr_context
        to_samples = config.FS / 1000
        f = rr_features(
            rc.rr_prev_ms * to_samples, rc.rr_next_ms * to_samples, (rc.rr_local_avg_ms or rc.rr_prev_ms) * to_
      samples
        )
        feats = torch.tensor([[f[c] for c in config.RR_FEATURES]], dtype=torch.float32)
        if getattr(models[name]["model"], "in_channels", 1) == 2:
            # the template is already a median of z-scored beats (as in training), so no re-normalising
            template_used = req.patient_template is not None
            tmpl = np.asarray(req.patient_template, dtype=np.float32) if template_used else x
            wf = np.stack([x, x - tmpl])[None]

    m = models[name]
    with torch.no_grad():
        raw = m["model"](torch.from_numpy(wf), feats).softmax(1)[0].double().numpy()
```

- **3** Return 503 if no model is loaded.
- **4–5** Z-score the beat exactly as in training.
- **8–15** If timing is given, use the Hybrid. Convert ms to samples and reuse the **same rr_features** function as training.
- **16–20** Second channel = beat − patient template (or zero difference if none is sent).
- **22–24** Run the model without gradients and turn scores into probabilities.

example

```
POST /predict  {"waveform": [280 numbers],
  "rr_context": {"rr_prev_ms": 620,
     "rr_next_ms": 1050, "rr_local_avg_ms": 830}}

-> "predicted_class": "S",
   "prediction_set": ["N", "S"], ...
```

Every response carries the header X-Disclaimer: Research demonstration only. Not for clinical use. Logs keep only a hash of the waveform.

8 · TRUST & SERVING

Dashboard, Docker and tests

Streamlit dashboard

Data Explorer: class counts, typical shapes, feature distributions, quality report.

Model Performance: all metrics, curves, ablation and per-patient results.

Live Prediction: draw a test beat or paste your own; see each model's answer, the prediction set, and which samples drove the decision (saliency).

Docker

One command starts the API and the dashboard in containers:

```
docker compose up --build
```

CPU-only PyTorch, runs as a non-root user, health checks included. The container reproduces the GPU-trained results exactly on all 400 demo beats.

82 automatic tests

```
pytest tests/ -q
```

- the filter doesn't move R-peaks
- windows are 280 samples
- labels map correctly
- broken data is caught
- models load and predict
- the API rejects bad input
- logs never contain the waveform

Reproducibility: one seed (42) for Python, NumPy and PyTorch; data and model files are fingerprinted with SHA-256 in training_metadata.json and evaluation_metrics.json, so you can prove which data produced which model.

WRAP-UP

How the project improved, iteration by iteration

#	Change	Effect
0	Pipeline as specified: LogReg/XGBoost on 13 features, CNN on the waveform	CNN 0.41 macro-F1 on unseen patients
1	Bug fix: the validation split had taken 90% of F beats out of training	F beats in training: 37 → 380
2	Patient-normalised RR features, Hybrid CNN, augmentation	LogReg 0.51 → 0.59; Hybrid ~0.62
3	Patient template channel (beat − usual beat)	Hybrid-rel 0.64–0.66 on DS2 (3 seeds)
4	5-fold patient-grouped CV for the final choice; 3-seed ensemble refit on all DS1	0.603 macro-F1, AUC 0.963 (selected without DS2)
5	Calibration + conformal prediction sets	Log-loss 0.477 → 0.158; 86% set coverage

Each step is recorded with numbers in docs/EXPERIMENTS.md, including the steps that did not help, so the final model's choice can be checked by anyone.

WRAP-UP

Limitations and what to try next

Limitations (be honest about them)

- Small, old dataset: 47 people recorded in 1975–79
- One lead (MLII) instead of the 12 a cardiologist uses
- Uses expert-marked R-peak positions; a real system needs its own beat detector
- F beats come almost entirely from one patient
- S precision is low (many false alarms)
- Not tested on any other hospital's data
- Not a medical device: research and learning only

Next steps (from docs/FUTURE_WORK.md)

1. Test on a second database (INCART) to see if results transfer
2. Learn the patient comparison (Siamese / prototype networks)
3. Sequence model over many beats (rhythm end to end)
4. Focal loss and patient-balanced sampling for fewer false alarms
5. MiniRocket: a fast, strong time-series baseline
6. Plug in a QRS detector for true end-to-end use
7. Self-supervised pre-training on large unlabelled ECG sets

WRAP-UP

Cheat sheet

ECG	recording of the heart's electrical activity
RR interval	time between two R-peaks
DS1 / DS2	train / test patient groups (de Chazal split)
Z-score	rescale to mean 0, spread 1
Epoch	one pass over all training data
Augmentation	random realistic changes to training beats
Cross-validation	rotate which patients are held out, average
Calibration	make 80% confidence mean right 80% of the time
R-peak	the tall spike of each beat; our anchor point
AAMI EC57	standard that defines the N/S/V/F classes
Band-pass filter	keeps only frequencies in a range (0.5–40 Hz)
Feature	a number describing a beat (e.g. rr_ratio)
Class weights	make mistakes on rare classes cost more
Early stopping	keep the epoch that did best on held-out patients
Ensemble	average several models' answers
Prediction set	all classes still plausible at 90% coverage

WRAP-UP

Recap

Data	MIT-BIH, 44 records, 100,630 beats, split by patient so tests are honest.
Pipeline	Filter → cut 280-sample windows → z-score → 16 features + patient template → checks.
Models	LogReg, XGBoost, CNN, and the winning Hybrid-rel ensemble (shape + timing, both patient-relative).
Results	Macro-F1 0.603, AUC 0.963, 96% of V and 84% of S beats found on unseen people.
Trust	Calibrated percentages, 90% prediction sets, every limitation documented.
Use it	API + dashboard + Docker + 82 tests; every command is on page 9.

More detail: README.md · docs/METHODOLOGY.md · docs/EXPERIMENTS.md · docs/API.md · docs/FUTURE_WORK.md · docs/model_card.md